

**Aluno:** Victor Casagrande

**Projeto:** Centro de Controle Logístico de Farmácias (Matriz e Filiais)

## 1. Estratégia Escolhida

A estratégia de comunicação implementada neste projeto foi o **WebSocket**, construída utilizando Node.js nativo com a biblioteca **ws** para instanciar o servidor.

## 2. Por que ela é adequada ao sistema

O sistema desenvolvido simula uma rede de farmácias onde filiais solicitam medicamentos e a matriz gerencia o estoque central e a logística, essa regra de negócio exige que o painel da matriz reflita novos pedidos instantaneamente e que as filiais sejam notificadas no exato momento em que uma entrega entra em trânsito, sendo assim, o WebSocket atende esse cenário, pois mantém um canal aberto que permite o envio imediato e o *broadcast* de atualizações de estoque e alertas visuais no mapa geográfico, sem que os clientes precisem atualizar a página manualmente ou fazer consultas repetitivas.

## 3. Diferenças entre as abordagens

**HTTP Polling:** O cliente faz requisições contínuas ao servidor em intervalos de tempo programados, mesmo que não haja nenhuma alteração nos dados do servidor, a requisição trafega pela rede, o que gera consumo desnecessário de banda e processamento.

**Long Polling:** O cliente envia a requisição e o servidor a "segura" aberta até que ocorra uma atualização real nos dados ou o tempo limite seja atingido, quando a resposta chega, o cliente a processa e imediatamente abre uma nova conexão, reduzindo as respostas vazias do Polling comum, mas ainda exige a reconstrução constante da conexão HTTP

**WebSocket:** Estabelece um único túnel TCP bidirecional logo no primeiro contato, que após aberto, tanto o cliente quanto o servidor podem enviar e receber mensagens livremente, permitindo atualizações instantâneas para múltiplos clientes sem o custo excessivo de cabeçalhos HTTP.

## 4. Vantagens e Desvantagens da escolhida

**Vantagens:** Proporciona latência quase nula, sendo a melhor opção para operações críticas em tempo real, reduzindo drasticamente o tráfego de rede e o *overhead* de processamento, além de facilitar a implementação de lógicas de *broadcast*.

**Desvantagens:** Como a conexão é mantida aberta, ela consome recursos de memória contínuos no servidor, além do dimensionamento horizontal ser significativamente mais complexo de configurar do que em APIs REST tradicionais, e conexões persistentes podem sofrer quedas frequentes em redes móveis instáveis ou ser bloqueadas por *proxies* corporativos rigorosos.

## 5. Quando NÃO usar essa estratégia

Não usaria essa estratégia em aplicações cujo fluxo de dados é eventual ou unidirecional como operações de CRUD simples (envio de formulários estáticos,

carregamento de portfólios ou leitura de blogs), pois em cenários onde a informação muda pouco ou não necessita ser imediata, o custo computacional de manter portas TCP abertas ociosas no servidor não se justifica.